CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 3 – CO3

Submitted by
Abinekavarshini.M
Register Number: 1924260333

Topics: Logical agents, propositional logic, resolution, first-order logic, unification, forward chaining and backward chaining.

Prepared as a task-wise academic answer guide based on the supplied assignment sheet.

Important note about experimental evidence

The assignment asks for measured execution times, counts, raw test data, plots and outputs from implementations. Those values depend on the exact code, computer and test knowledge bases. This report gives reproducible methods, sample knowledge bases, algorithms and code templates. It does not present invented timings or claim that unexecuted experiments were measured. Run the programs and replace the clearly identified sample/expected entries with your actual outputs before submission.

For Task 7, the subject is medication safety. The proposed system is an educational decision-support prototype only; it must not make real clinical decisions. Real deployment requires validated clinical rules, privacy controls, clinician oversight and testing.

TASK 1 – Logical Agents: Knowledge Bases, Entailment and Reasoning Patterns (6 Marks)

1.1 Wumpus-world knowledge base

For each cell (i,j), define propositional symbols Pij = “there is a pit”, Wij = “there is a Wumpus”, Bij = “there is a breeze” and Sij = “there is a stench”. Let N(i,j) be the set of orthogonally adjacent cells inside the n × n grid.

The percept rules are: Bij ↔ OR of Pkl for all (k,l) in N(i,j), and Sij ↔ OR of Wkl for all (k,l) in N(i,j). Also add the domain constraints: exactly one Wumpus (if required by the chosen world), at most one Wumpus per cell, and start cell (1,1) is safe. A cell is safe when ¬Pij ∧ ¬Wij.

A biconditional A ↔ (X1 ∨ … ∨ Xd) can be encoded as (¬A ∨ X1 ∨ … ∨ Xd) and, for every r, (¬Xr ∨ A). Boundary cells have fewer neighbours.

1.2 Symbol and model counts

There are $4n^2$ basic proposition symbols if all four predicates are represented for every cell. Thus $m = 4n^2$ and an unrestricted truth-table checker has $2^m = 2^{(4n^2)}$ possible assignments. If auxiliary propositions or extra rules are introduced, include them in m.

Grid n	Basic symbols m	All assignments 2^m
2	16	65,536
3	36	68,719,476,736
4	64	18,446,744,073,709,551,616

The count of KB sentences depends on the encoding. With one percept biconditional per cell for breeze and one for stench, there are $2n^2$ biconditionals before expanding them into clauses, plus the safety/start and world-constraint sentences. For a query about one cell, retain the query cell, its percept symbols, neighbouring hazard symbols and the rules connecting them; recursively include only facts/rules that can affect those symbols. This is the relevant-symbol restriction.

1.3 Truth-table entailment checker

KB entails α exactly when every model satisfying KB also satisfies α. A direct checker enumerates assignments and tests whether KB ∧ ¬α is satisfiable. If no such model exists, entailment holds.

```
from itertools import product

def entails(symbols, clauses, query):
    # clauses are lists of (symbol, required_boolean) literals.
    # query is a (symbol, required_boolean) pair.
    models = kb_models = 0
    countermodels = []
    for bits in product([False, True], repeat=len(symbols)):
        model = dict(zip(symbols, bits))
        models += 1
        ok = all(any(model[s] == val for s, val in clause)
                 for clause in clauses)
        if ok:
            kb_models += 1
            if model[query[0]] != query[1]:
```

```
                countermodels.append(model)
    return not countermodels, models, kb_models, countermodels
```

The code above uses a CNF clause representation. A practical implementation should use a monotonic clock (time.perf_counter) around the enumeration and record assignments visited, KB evaluations and elapsed seconds. Relevant-symbol restriction reduces the number of assignments from 2^m to 2^r, where r is the number of retained symbols.

1.4 Three query outcomes

Query condition	Correct interpretation
KB ■ α	Entailed: α is true in every model of KB.
KB ■ ¬α	Negation entailed: α is false in every model of KB.
Neither KB ■ α nor KB ■ ¬α	Undetermined: models exist with α true and with α false.

For a reproducible Wumpus example, assume the start cell has no breeze and no stench. The adjacent cells (1,2) and (2,1) are then pit-free and Wumpus-free under the stated adjacency rules. A square not constrained by any percept may remain undetermined. State the exact percept facts used with each query.

1.5 Reasoning patterns and soundness

Modus Ponens: from P and P → Q, infer Q. **And-Elimination:** from P ∧ Q, infer P (or Q). **Unit Resolution:** from (P ∨ Q) and ¬P, infer Q.

Truth-table soundness check: in every row where all premises of a rule are true, its conclusion is also true. Therefore, each rule preserves truth in every model. By induction over a sequence of sound inference steps, every derived sentence is true in every model of the original KB. “Not derived” does not mean false: the inference procedure may be incomplete, or the fact may simply not follow from the available information.

P	Q	P → Q	MP conclusion Q when P and P→Q true
T	T	T	T
T	F	F	Premises not both true
F	T	T	Premises not both true
F	F	T	Premises not both true

For the same query, compare model checking and rule-based inference by counting assignments checked versus inference steps. Report measured counts from your implementation; the expected qualitative result is that local rule derivations can avoid exploring unrelated assignments.

TASK 2 – Model Checking versus Propositional Resolution (7 Marks)

2.1 Example smart-building access policy

Symbols: V = badge valid, D = daytime, E = employee, G = registered guest, H = host approval, M = emergency override, A = access granted, L = restricted-area training completed, X = access suspended.

A possible policy knowledge base (at least ten rules) is:

1. $V \wedge D \rightarrow A$ (valid badge during daytime permits access).

2. $E \wedge L \rightarrow A$ (trained employee is eligible).

3. $G \wedge H \rightarrow A$ (registered guest with host approval is eligible).

4. $M \rightarrow A$ (emergency override permits access, subject to authorised emergency policy).

5. $X \rightarrow \neg A$ (suspension denies access).

6. $A \rightarrow (V \vee (E \wedge L) \vee (G \wedge H) \vee M)$ (every grant has a recognised basis).

7. $V \leftrightarrow$ BadgeActive (badge validity matches active badge status).

8. $G \rightarrow$ VisitorRecordExists.

9. $H \rightarrow$ HostIsEmployee.

10. $L \rightarrow$ SafetyTrainingPassed.

11. $X \rightarrow$ ReviewRequired.

12. $\neg D \wedge \neg M \rightarrow \neg A$ (outside daytime, no emergency override, access is denied).

The rules include implication, biconditional and disjunction. The policy is an illustrative logic exercise, not a real access-control policy.

2.2 CNF conversion examples

Use $A \rightarrow B \equiv \neg A \vee B$; $A \leftrightarrow B \equiv (\neg A \vee B) \wedge (\neg B \vee A)$; and distribute $\vee$ over $\wedge$.

Original sentence	CNF
$V \wedge D \rightarrow A$	$\neg V \vee \neg D \vee A$
$G \wedge H \rightarrow A$	$\neg G \vee \neg H \vee A$
$V \leftrightarrow$ BadgeActive	$(\neg V \vee$ BadgeActive$) \wedge (\neg$BadgeActive $\vee V)$

2.3 Resolution refutation

To prove KB ■ Q, add ¬Q to the CNF KB and derive the empty clause ■. The resolution rule is: from $(C \vee P)$ and $(D \vee \neg P)$, infer $(C \vee D)$. A contradiction proves entailment.

Example: clauses $(\neg V \vee A)$, V, and ¬A. Resolve $(\neg V \vee A)$ with V to obtain A; resolve A with ¬A to obtain ■. Hence the KB entails A.

2.4 Comparison and experiments

Measure	Truth-table model checking	Resolution
Main work	Enumerates assignments	Generates resolvents
Strong case	Small number of symbols; easy countermodel search	Sparse clauses and short proof chains
Weak case	Large symbol count (2^m assignments)	Large clause growth and many possible resolvents
Evidence	Assignments, KB evaluations, time	Clauses, attempted resolvents, tautologies, subsumption, time

Build at least two KB families: (i) a small-symbol family with many irrelevant clauses, where direct model enumeration may be practical; (ii) a sparse implication-chain family, where resolution can follow a short proof path. For chain lengths 10, 20, 40 and 80, record the actual generated clauses and elapsed time. A non-entailed query does not produce the empty clause; a complete resolution procedure eventually reaches saturation (no new clauses) or detects that the search limit was reached.

Joint design answer: Before choosing, inspect the number of symbols, number and width of clauses, clause connectivity, expected query type, whether a proof or countermodel is needed, KB update frequency, memory budget and completeness requirements. Benchmark representative KB families rather than relying on one example. Model checking is attractive for small finite vocabularies; resolution is attractive when a proof can be found through a manageable set of relevant clauses.

TASK 3 – Knowledge Engineering in First-Order Logic (7 Marks)

3.1 University exam-eligibility advisor

Knowledge-engineering stages: identify the task; collect regulations; choose vocabulary; encode general rules; add student/course facts; ask queries; debug and validate the KB.

Predicates: Student(x), Course(c), Attendance(x,c,p), PrerequisiteCleared(x,c), FeesPaid(x,c), DisciplinaryHold(x), ApprovedLeave(x,c), Eligible(x,c). Use p as an attendance percentage.

3.2 First-order axioms

1. $\forall x \forall c\ [(\text{Student}(x) \wedge \text{Course}(c) \wedge \text{Attendance}(x,c,p) \wedge p \geq 75 \wedge \text{PrerequisiteCleared}(x,c) \wedge \text{FeesPaid}(x,c) \wedge \neg\text{DisciplinaryHold}(x)) \rightarrow \text{Eligible}(x,c)]$.

2. $\forall x \forall c\ [\text{ApprovedLeave}(x,c) \rightarrow \text{LeaveMayWaiveAttendance}(x,c)]$.

3. $\forall x \forall c\ [(\text{LeaveMayWaiveAttendance}(x,c) \wedge \text{Student}(x) \wedge \text{Course}(c) \wedge \text{PrerequisiteCleared}(x,c) \wedge \text{FeesPaid}(x,c) \wedge \neg\text{DisciplinaryHold}(x)) \rightarrow \text{Eligible}(x,c)]$.

4. $\forall x \forall c\ [\text{Eligible}(x,c) \rightarrow \text{Student}(x) \wedge \text{Course}(c)]$.

5. $\forall x \forall c\ [\text{DisciplinaryHold}(x) \rightarrow \neg\text{Eligible}(x,c)]$.

6. $\forall x \forall c\ [\neg\text{FeesPaid}(x,c) \rightarrow \neg\text{Eligible}(x,c)]$.

7. $\forall x \forall c\ [\neg\text{PrerequisiteCleared}(x,c) \rightarrow \neg\text{Eligible}(x,c)]$.

8. $\forall x \forall c \forall p\ [(\text{Attendance}(x,c,p) \wedge p < 75 \wedge \neg\text{ApprovedLeave}(x,c)) \rightarrow \neg\text{Eligible}(x,c)]$.

9. $\forall x \forall c\ [\text{ApprovedLeave}(x,c) \rightarrow \text{Student}(x) \wedge \text{Course}(c)]$.

10. $\forall x \forall c\ [\text{PrerequisiteCleared}(x,c) \rightarrow \text{Student}(x) \wedge \text{Course}(c)]$.

11. $\forall x \forall c\ [\text{FeesPaid}(x,c) \rightarrow \text{Student}(x) \wedge \text{Course}(c)]$.

12. $\forall x\ [(\text{Student}(x) \rightarrow \exists c\ \text{Course}(c))]$.

The existential axiom states that every student has at least one course in the domain. In a real KB, define precisely whether approved leave waives attendance and whether the 75% threshold is inclusive.

3.3 Propositional versus FOL growth

If there are s students and c courses, a propositional encoding typically creates student-course-specific facts and rules, so its instance-specific sentence count grows on the order of O(sc) (plus student-only and course-only facts). The FOL representation stores general quantified rules once; its axiom count is O(1) with respect to s and c, while the number of ground facts still grows with the data, typically O(sc).

Greater expressiveness does not guarantee faster inference. FOL requires unification and may search over substitutions; a compact rule set can still produce expensive inference. Propositionalization may be faster for a small, fixed domain.

3.4 Competency query suite

Type	Example query	Expected result for a suitable test KB
Success	Eligible(Anu,Physics)?	True if all conditions are met
Success	Eligible(Ravi,Chemistry)?	True if approved leave validly waives attendance and other conditions h
Failure	Eligible(Devi,Maths)?	False if fees are unpaid
Failure	Eligible(Kumar,AI)?	False if prerequisite is not cleared
Bindings	Eligible(x,Physics)?	Returns each eligible student x
Bindings	FeesPaid(x,c)?	Returns student-course substitutions

Expand this pattern into at least 15 distinct tests: five successful, five failing and five binding-return queries. Record the input facts, expected answer, actual answer and pass/fail. Typical defects to test include a missing negated disciplinary-hold condition, using $p > 75$ instead of $p \geq 75$, and accidentally allowing unpaid fees through the approved-leave exception. Fix defects and retain a defect log.

Measure sentence count and query time at several increasing values of s and c. Keep hardware, software, query set and timing method constant. Do not claim a crossover point until it is observed in the actual measurements.

TASK 4 – Unification and Lifted Inference (7 Marks)

4.1 Unification

Unification finds a substitution θ that makes two expressions identical. The most general unifier (MGU) makes the fewest commitments while unifying the expressions. Substitution composition applies one substitution and then another consistently.

Pair of terms	MGU / result
Knows(x,Abi), Knows(Ravi,y)	{x/Ravi, y/Abi}
Parent(x,John), Parent(Mary,y)	{x/Mary, y/John}
Likes(x,x), Likes(Abi,y)	{x/Abi, y/Abi}
P(f(x),y), P(f(Abi),z)	{x/Abi, y/z}
R(x), R(f(x))	Fails occurs check: x cannot equal f(x)

Standardising apart is essential when combining rules: rename variables in one rule so that variables from different rule applications are independent. For example, use Parent(u,v) → Ancestor(u,v) with fresh u,v rather than accidentally reusing variables from another clause.

4.2 Unification implementation

```
def unify(x, y, theta=None):
    if theta is None:
        theta = {}
    x = walk(x, theta)
    y = walk(y, theta)
    if x == y:
        return theta
    if isinstance(x, str) and x.startswith("?"):
        if occurs(x, y, theta): return None
        theta[x] = y
        return theta
    if isinstance(y, str) and y.startswith("?"):
        if occurs(y, x, theta): return None
        theta[y] = x
        return theta
    if isinstance(x, tuple) and isinstance(y, tuple):
        if len(x) != len(y) or x[0] != y[0]:
            return None
        for a, b in zip(x[1:], y[1:]):
            theta = unify(a, b, theta)
            if theta is None: return None
        return theta
    return None

def walk(x, theta):
    while isinstance(x, str) and x in theta:
        x = theta[x]
    return x

def occurs(v, term, theta):
    term = walk(term, theta)
    if v == term: return True
    if isinstance(term, tuple):
        return any(occurs(v, item, theta) for item in term[1:])
    return False
```

Represent constants as strings without a leading question mark and variables with a leading ?. Represent compound terms as tuples such as ('f','?x'). The code is a compact teaching implementation; a production prover should also recursively apply substitutions to nested terms and implement robust variable standardisation-apart and substitution composition.

4.3 Lifted Modus Ponens

Given facts/rules whose premises unify with known facts, compute the MGU and apply the resulting substitution to the conclusion. For example, from Parent(x,y) ∧ Parent(y,z) → Grandparent(x,z), and facts Parent(Ana,Bob), Parent(Bob,Cam), unification gives x = Ana, y = Bob, z = Cam; infer Grandparent(Ana,Cam).

Lifted inference is sound because it applies a sound inference rule using a substitution that makes the premises match. It can avoid generating every possible ground instance: it instantiates only variables needed for a successful match.

4.4 Grounding growth and engineering decision

For a predicate of arity a over k constants, there are k^a possible ground tuples. For example, a binary predicate has k^2 ground instances: 9 when k=3 and 900 when k=30. A unary predicate has k instances. With several predicates, sum the possibilities for each predicate.

Constants k	Unary tuples k	Binary tuples k^2	Ternary tuples k^3
3	3	9	27
10	10	100	1,000
30	30	900	27,000

If function symbols are allowed, term counts can grow rapidly with depth. With one constant a and one unary function f, terms up to depth d are a, f(a), …, f^d(a): d+1 terms. With multiple function symbols or higher arity functions, the number grows faster. Function symbols can create an unbounded Herbrand universe.

Propositionalize when the domain is small and finite, all relevant constants are known, ground clauses can be indexed efficiently, and a simple SAT/resolution engine performs better in measured tests. Prefer lifted inference when the domain is large or changing and general rules can be reused without creating all ground instances.

TASK 5 – First-Order Resolution (6 Marks)

5.1 Convert the statements to FOL

Predicates: Employee(x), Handled(x,d), Restricted(d), PatientData(d), Audited(d), Authorised(x). The four statements become:

• $\forall x \forall d\ [(\text{Employee}(x) \wedge \text{Handled}(x,d) \wedge \text{Restricted}(d)) \rightarrow \text{Authorised}(x)]$.

• $\forall d\ [\text{PatientData}(d) \rightarrow \text{Restricted}(d)]$.

• $\exists x\ [\text{Employee}(x) \wedge \forall d\ (\text{Audited}(d) \rightarrow \text{Handled}(x,d))]$.

• $\text{PatientData}(R1) \wedge \text{Audited}(R1)$.

5.2 CNF and Skolemisation

1. $\neg\text{Employee}(x) \vee \neg\text{Handled}(x,d) \vee \neg\text{Restricted}(d) \vee \text{Authorised}(x)$.

2. $\neg\text{PatientData}(d) \vee \text{Restricted}(d)$.

3. Employee(c) (Skolem constant c witnesses the existential employee).

4. $\neg\text{Audited}(d) \vee \text{Handled}(c,d)$.

5. PatientData(R1). 6. Audited(R1).

The existential variable is not inside the scope of a universal variable, so a Skolem constant c is sufficient; no Skolem function is needed. Skolemisation preserves satisfiability, not logical equivalence: it replaces an existential witness with a fresh symbol in an expanded language.

5.3 Resolution proof that an authorised employee exists

From clauses 5 and 2, resolve on PatientData(R1) to obtain Restricted(R1). From clauses 6 and 4, resolve on Audited(R1) to obtain Handled(c,R1). With Employee(c) from clause 3, and Restricted(R1), use clause 1 with x=c and d=R1 to derive Authorised(c). Therefore $\exists x$ Authorised(x).

For refutation, negate the goal: $\neg\exists x$ Authorised(x), equivalent to $\forall x \neg$Authorised(x). Add ¬Authorised(c). Resolving it with Authorised(c) produces the empty clause ■, proving the goal.

5.4 Strategy and non-entailed query

A set-of-support strategy selects at least one parent clause from the negated goal or its descendants for every resolution step. Unit preference prioritises clauses containing one literal. Record clauses generated, resolution steps, proof length and unifications for the chosen strategy and for an unprioritised baseline.

A suitable second query is “Is Employee(Alice) entailed?” when no fact or rule identifies Alice. It is not entailed by the given facts. First-order resolution may not terminate for arbitrary non-entailed queries; a step cap is an engineering safeguard, not a proof of non-entailment. Report “no proof found within the cap”, not “false”. Compare against a finite propositional grounding of a small domain.

TASK 6 – Forward and Backward Chaining (7 Marks)

6.1 Organisation hierarchy KB

Use predicates ReportsTo(x,y), Manager(x), Approves(x,r), and CanApprove(x,r). Example definite-clause rules:

• ReportsTo(x,y) ∧ Manager(y) → ReportsToManager(x,y).

• ReportsTo(x,y) ∧ Approves(y,r) → CanApprove(x,r).

• Manager(x) ∧ Approves(x,r) → CanApprove(x,r).

• ReportsTo(x,y) ∧ ReportsTo(y,z) → ReportsToIndirectly(x,z).

• ReportsToIndirectly(x,y) ∧ Manager(y) → ReportsToManager(x,y).

• CanApprove(x,r) → ApprovalEligible(x,r).

• Manager(x) → ApprovalEligible(x,r) (only if policy explicitly grants this broad authority).

• ReportsTo(x,y) ∧ ApprovalEligible(y,r) → ApprovalEligible(x,r) (only if delegated authority is intended).

• ReportsTo(x,y) ∧ ReportsTo(y,z) → ReportsToIndirectly(x,z).

• Approves(x,r) → CanApprove(x,r).

Use a consistent predicate arity in the actual implementation. The examples marked as policy-dependent should be included only if the organisation's rules explicitly support them.

6.2 Forward chaining

Forward chaining is data-driven. Start with known facts, repeatedly apply rules whose premises match known facts, add new conclusions, and stop when no new facts can be added (fixed point).

```
def forward_chain(facts, rules):
    known = set(facts)
    changed = True
    firings = 0
    while changed:
        changed = False
        for premises, conclusion in rules:
            # match_rule returns all substitutions satisfying premises
            for theta in match_rule(premises, known):
                new_fact = substitute(conclusion, theta)
                firings += 1
                if new_fact not in known:
                    known.add(new_fact)
                    changed = True
    return known, firings
```

Measure facts inferred, rule firings, redundant derivations, iterations to fixed point and the percentage of derived facts irrelevant to a selected query. If a query concerns a small region of a large KB, forward chaining may still derive many unrelated consequences.

6.3 Backward chaining

Backward chaining is goal-driven. Begin with the query, find rules whose conclusion unifies with it, and recursively prove their premises as subgoals. It can avoid exploring facts unrelated to the query.

```
def prove(goal, rules, facts, visited=None, depth=0, limit=100):
    if visited is None:
        visited = set()
    if goal in facts:
        return True
    if depth >= limit or goal in visited:
        return False
    visited = visited | {goal}
    for premises, conclusion in rules:
        theta = unify_atoms(conclusion, goal)
        if theta is not None:
            if all(prove(apply(p, theta), rules, facts,
                         visited, depth + 1, limit)
                   for p in premises):
                return True
    return False
```

The pseudocode assumes helper functions for atom unification and substitution. A left-recursive rule such as Reach(x,z) ← Reach(x,y) ∧ Edge(y,z) may cause naive depth-first backward chaining to revisit the same goal indefinitely. A loop check, depth bound or memoisation can prevent unbounded repetition. A depth bound can miss valid long proofs; loop checking can be incomplete in some settings; memoisation uses extra memory.

6.4 Verification and comparison

Aspect	Forward chaining	Backward chaining
Direction	Facts → consequences	Goal → supporting subgoals
Best suited	Many queries or need to materialise consequences	Few specific queries
Potential cost	May derive irrelevant facts	May repeat subgoals or loop without control
Metrics	Facts, firings, iterations, irrelevant percentage	Goals, unifications, depth, repeated subgoals

Run the same KB at 100, 1,000 and 10,000 facts. Keep rules and query constant, then record the requested metrics. Backward chaining can remain efficient when many facts are irrelevant because it follows only rules and facts that can support the requested goal, provided indexing and loop control are effective.

TASK 7 – Hospital Medication-Safety Advisor (10 Marks)

7.1 Stage 1: independent design proposals

Student A proposal: Use first-order logic because the system must represent many patients, medicines, allergies and conditions with reusable general rules. Use backward chaining for a specific prescribed-drug query and produce an explanation by recording the rules and facts used.

Student B proposal: Use a rule-based FOL knowledge base with indexed patient and medicine facts. Use forward chaining to precompute high-priority contraindication flags, then backward chaining to answer a requested prescription query. Explain decisions by listing matched rules and missing or conflicting evidence.

7.2 Stage 2: compare designs

Design	Strength	Limitation
A: FOL + backward chaining	Focused query; reusable rules; traces relevant proof	Repeated queries may repeat work; depends on rule indexing
B: FOL + forward/backward hybrid	Precomputes important alerts and supports targeted explanation	More updates and consistency complexity

Joint decision: retain FOL, indexed predicates and backward chaining as the primary query method. Use a controlled forward-chaining process for updating validated alert facts when patient records change. Keep a complete proof/explanation trace.

7.3 Stage 3: final design

Vocabulary: Patient(p), Drug(d), HasAllergy(p,a), ContainsIngredient(d,a), Takes(p,m), Interaction(d,m), HasCondition(p,c), Contraindicated(d,c), Age(p,n), MinimumAge(d,n), RequiresReview(d), SafeToPrescribe(p,d), Alert(p,d,reason).

Core rules:

• HasAllergy(p,a) ∧ ContainsIngredient(d,a) → Alert(p,d,allergy_conflict).

• Takes(p,m) ∧ Interaction(d,m) → Alert(p,d,drug_interaction).

• HasCondition(p,c) ∧ Contraindicated(d,c) → Alert(p,d,condition_conflict).

• Age(p,n) ∧ MinimumAge(d,min) ∧ n < min → Alert(p,d,age_restriction).

• RequiresReview(d) → Alert(p,d,clinician_review_required).

• Alert(p,d,r) → ¬SafeToPrescribe(p,d).

• Patient(p) ∧ Drug(d) ∧ CompleteRecord(p) ∧ ¬∃r Alert(p,d,r) → CandidateForReview(p,d).

The final rule is intentionally named CandidateForReview, not an unconditional clinical approval. Absence of a known alert does not prove safety when records or knowledge may be incomplete.

Indexing/agenda: Index facts by patient ID, drug ID, ingredient, condition and interaction pair. Use a worklist agenda to process only newly added facts and affected rules. Cache query results with version identifiers and invalidate them when patient or rule data changes.

Explanation mechanism: Store each derived alert with its rule ID, matched facts, substitutions, timestamp and knowledge-base version. Return a plain-language explanation showing the triggering facts and the rule that fired. If evidence is missing, return “insufficient information” rather than assuming the condition is absent.

7.4 Complete high-level pseudocode

```
function assessPrescription(patient, drug, KB):
    verify patient and drug identifiers
    load current patient facts and validated drug rules
    if required patient data are missing:
        return INSUFFICIENT_INFORMATION with missing fields

    agenda = facts relevant to patient and drug
    alerts = []
    while agenda is not empty:
        fact = pop(agenda)
        for rule in indexedRulesAffectedBy(fact):
            for substitution in unify(rule.premises, knownFacts):
                if all premises are satisfied:
                    conclusion = apply(rule.conclusion, substitution)
                    if conclusion is new:
                        store conclusion and its proof trace
                        push conclusion onto agenda

    alerts = all Alert(patient, drug, reason) conclusions
    if alerts is not empty:
        return REVIEW_REQUIRED, alerts, proof traces
    else:
        return NO_KNOWN_ALERTS_REQUIRES_CLINICIAN_REVIEW
```

7.5 Stage 4: validation plan

Use at least 30 test cases, including single and combined conditions. Example test categories: known allergy to an ingredient; interaction with a current medicine; contraindicated condition; age threshold; multiple simultaneous alerts; missing allergy list; incomplete medication list; unknown drug ingredient; duplicate facts; contradictory facts; and a no-known-alert case. Include both positive and negative controls.

Validation scale	Required measurements
40 rules	Baseline query latency, inference steps, memory and explanation length
400 rules	Same metrics; check indexing benefit
2,000 rules	Same metrics; stress test updates and concurrent patient records

Use identical test cases at each KB size. Report median and 95th-percentile query latency, number of rules examined, unifications, peak memory and explanation length. Compare indexed inference with a simple unindexed baseline. Do not invent values; collect them from actual runs. Include dangerous combinations such as allergy plus interacting medicine and contraindicated condition plus missing records.

7.6 Stage 5 and Stage 6: validation and evaluation

Theoretical validation: A sound rule engine derives only conclusions supported by its rules and facts. Completeness depends on the chosen rule language and inference procedure; a restricted definite-clause program supports efficient goal-directed reasoning under suitable conditions. Query complexity depends on rule structure, indexing, data size and recursion; worst-case reasoning can still be expensive.

Experimental validation: Use the 30+ test cases and three KB sizes above. Retain raw outputs and scripts. Compare the proposed indexed hybrid with the baseline using the same hardware and input data.

Why a logical agent? The task has explicit facts, rules and explainable conditions. Logic can provide traceable reasons for alerts. **Limits:** uncertainty, missing or outdated records, ambiguous drug data, dosage/context effects and unknown interactions cannot be safely resolved by simple true/false rules. **Before choosing:** examine rule quality,

completeness, update frequency, scale, explanation needs, uncertainty handling, safety requirements, privacy, latency and clinician oversight.

Contribution Record (complete truthfully with your partner)

The assignment requires both students to contribute to every task and understand the full submission. Fill in this table according to the work actually completed; do not claim contributions that were not made.

Task	Student A – Primary contribution	Student B – Primary contribution	Joint work
1	Design / mathematical analysis	Experimental validation	KB, reasoning and comparison
2	Resolution investigation	Model-checking analysis	Policy and design decision
3	FOL design derivation	Crossover investigation	Knowledge engineering
4	Experimental engineering analysis	Unification correctness	Lifted inference
5	Conversion and proof design	Comparative evidence	FOL resolution prover
6	Forward chaining	Backward chaining	Shared KB and verification
7	As agreed by pair	As agreed by pair	Final design and evaluation

Final checklist before submission

- ■ Add the submission date and partner's details if required.
- ■ Run and test every program; include source code and actual outputs.
- ■ Replace illustrative examples with the exact KB and percept facts used in your experiments.
- ■ Add measured tables/graphs for model counts, clauses, inference steps, query time and memory.
- ■ Include the 15 FOL competency queries and 30 or more Task 7 test cases.
- ■ Complete the contribution record honestly.
- ■ Prepare to explain both your own and your partner's sections during the viva.